



Technical Documentation

Root Filesystem Image Builder

Team Oriented Project

Group A

Linux Kernel Extension for DM-Verity Authentication, Verification and Setup

Date: **NOVEMBER 2025**

Cooperation with:

**BMW
GROUP**



ROLLS-ROYCE
MOTOR CARS LTD

[BMW Car IT, Ulm](#)

Eva-Helena Zorman

Tobias Kaufmann

Philipp Graf

1. Introduction

1.1. Purpose

This document provides technical specifications and operational details for the root filesystem image builder script (`build_rootfs.sh`). Its primary function is to generate a complete, single-disk, unpartitioned Debian-based root filesystem image (`rootfs.img`).

This image is a foundational component of a secure boot system, specifically engineered to accommodate Device-mapper Verity (`dm-verity`) integrity protection metadata. The resulting image is designed for deployment in environments where the entire filesystem's integrity must be cryptographically verified by the kernel during the early boot process.

1.2. Background and Project Context

This script was developed to meet a customer requirement to shift security and integrity responsibility from a secondary bootloader (e.g., U-Boot, GRUB) to the Linux kernel. This shift significantly enhances security by ensuring the kernel verifies the root filesystem before any user-space process can execute. The script's design focuses on creating the necessary slack space at the end of the image for the `dm-verity` hash tree and metadata, which will be appended by a subsequent script (`generate_verity.sh`).

2. Script Specification: `build_rootfs.sh`

2.1. Overview

The `build_rootfs.sh` script executes a four-stage build pipeline: dependency installation, system configuration, disk image creation, and file synchronization. It ensures a minimal, reproducible, and integrity-ready disk image.

2.2. Prerequisites

The host system requires `sudo` access and standard Linux utilities, including `debootstrap` (for base system installation), `rsync` (for file copying), and filesystem tools like `mkfs.ext4`, `truncate`, `mount`, and `umount`.

2.3. Build Process Details

The script executes the following four primary stages:

Stage [1/4]: Creating Base Debian Rootfs

This stage installs a minimal Debian Bookworm base system into the `$ROOTFS/_DIR` directory. The installation is performed using `debootstrap` with the `amd64` architecture.

The `--include` parameter ensures only essential packages necessary for a secure, networked, and manageable system are installed:

```
37 | | | | --include=systemd,systemd-sysv,udev,passwd,login,sudo,net-tools,iproute2,ifupdown,openssh-server,vim,less \
38 | | | | bookworm "$ROOTFS_DIR" http://deb.debian.org/debian/
```

Stage [2/4]: Configuring Rootfs

This stage mounts the necessary special filesystems (proc, sys, and dev) and uses a chroot environment to configure the base system.

- **User Account Setup:** Three distinct accounts are created for security and system management, adhering to the principle of Least Privilege: root, admin (added to the sudo group via `usermod -aG sudo admin`), and user.
- **System Configuration:** The hostname is set to `secureboot-demo`, and networking is configured for DHCP on `eth0` by modifying `/etc/network/interfaces`.

After configuration, the special filesystems are safely unmounted.

Stage [3/4]: Sizing Disk Image and Creating Filesystem

This stage calculates the final image size and creates the single ext4 filesystem.

- **dm-verity Slack Space Reservation:** The script estimates the space required by the current filesystem data (`ROOTFS_SIZE_MB`) and then calculates an essential `VERITY_SPACE_MB` (approximately 20% of the data size plus a buffer). This extra space is crucial and is calculated as:

```
117 VERITY_SPACE_MB=$(( (ROOTFS_SIZE_MB / 5 + 20) ))    # ~20% + 20 MB
118 DISK_SIZE_MB=$(( (ROOTFS_SIZE_MB + VERITY_SPACE_MB) ))
```

This reserved block of unallocated space at the end of the image is designated for the dm-verity hash tree and metadata.

- **Single Block Device Creation:** The total size is enforced using `truncate`, and the ext4 filesystem is created directly on the image file (`$OUTPUT_IMG`) using:
`mkfs.ext4 -F -L rootfs "$OUTPUT_IMG"`. Crucially, no partition table (GPT/MBR) is created.

Stage [4/4]: Copying Rootfs into Disk Image

The final stage mounts the newly created image to `$TEMP_MOUNT` and uses `rsync` to efficiently copy the entire prepared root filesystem tree into the ext4 structure.

```
142 echo " Copying files to filesystem..."
143 sudo rsync -aHAX --info=progress2 "$ROOTFS_DIR"/ "$TEMP_MOUNT"/
```

The mount is then safely unmounted (`sudo umount "$TEMP_MOUNT"`) to flush all data, finalizing the disk image.

3. Key Technical Decisions and Security Implications

3.1. Single Block Device Architecture

The decision to create the ext4 filesystem directly on the disk image, without a partition table (GPT/MBR), is intentional. This **simplifies the dm-verity verification step** because the kernel can treat the entire image as one contiguous verifiable block device. This choice also eliminates potential security risks associated with malicious manipulation of the partition metadata.

3.2. Dedicated dm-verity Slack Space

The most critical technical decision is the explicit reservation of space in Stage [3/4]. By calculating and reserving approximately 20% additional space (`VERITY_SPACE_MB`), the script guarantees sufficient contiguous unallocated blocks at the end of the image.

This space is reserved for the dm-verity generation tool (the subsequent `generate_verity.sh`) to place the complete **Merkle hash tree** and the **dm-verity metadata header**. This architecture ensures that the final image is perfectly sized for integrity verification, where the root filesystem data blocks are directly followed by their cryptographic verification data.